

Agents Lab

Samarth Bhatnagar

September 2026

Contents

Contents

1	Task 0: Understand the Planning Problem	2
2	Task 1: Construct a Plan by Hand	3
3	Task 2: Ask an LLM to Implement the Planner	4
4	Task 3: Test the Generated Planner	6
5	Task 4: Logic and Search	9
6	Task 5: Can the LLM Verify Its Own Plan?	10
7	Reflection Questions	11

1 Task 0: Understand the Planning Problem

The planning problem consists of a robot and a package located in a warehouse with three locations A , B , and C . The robot and package initially start at A , and the goal is to transport the package to C .

(a) Initial State

The initial state is

$$I = \{At(Robot, A), At(Package, A)\}.$$

Thus, both the robot and the package are initially at location A .

(b) Goal

The goal is

$$G = \{At(Package, C)\}.$$

Only the final location of the package is relevant to the goal.

(c) Available Actions

The robot has three types of actions:

- **Move**: move between connected locations;
- **PickUp**: pick up the package when the robot and package are at the same location;
- **Drop**: drop the package at the robot's current location.

The available movements are

$$Move(A, B), \quad Move(B, A), \quad Move(B, C), \quad Move(C, B).$$

(d) Preconditions and Effects

Action	Preconditions	Effects
$Move(X, Y)$	$At(Robot, X)$	$\neg At(Robot, X), At(Robot, Y)$
$PickUp(Package, X)$	$At(Robot, X), At(Package, X)$	$\neg At(Package, X), Holding(Package)$
$Drop(Package, X)$	$At(Robot, X), Holding(Package)$	$\neg Holding(Package), At(Package, X)$

Table 1: Actions, preconditions, and effects of the planning problem.

Initially Applicable Actions

The initial state is

$$I = \{At(Robot, A), At(Package, A)\}.$$

The action $PickUp(Package, A)$ has the preconditions

$$At(Robot, A)$$

and

$$At(Package, A).$$

Both preconditions are satisfied in the initial state. Therefore,

$PickUp(Package, A)$ is applicable.

The action $Drop(Package, C)$ requires

$$At(Robot, C)$$

and

$$Holding(Package).$$

Neither condition is satisfied in the initial state. Therefore,

$Drop(Package, C)$ is not applicable.

Think About It

An action should not be considered applicable merely because it appears in the list of available actions. All of its preconditions must be satisfied in the current state. In this problem, logical reasoning is used to determine whether the preconditions of an action are satisfied before the action is applied.

2 Task 1: Construct a Plan by Hand

The initial state is

$$S_0 = \{At(Robot, A), At(Package, A)\}.$$

According to the action definitions, an action can be applied only when all of its preconditions are satisfied.

The laboratory gives the following example sequence:

$$Move(A, B), \quad PickUp(Package, B), \quad Move(B, C), \quad Drop(Package, C).$$

However, applying the first action to the stated initial state gives

$$S_0 \xrightarrow{Move(A,B)} S_1$$

where

$$S_1 = \{At(Robot, B), At(Package, A)\}.$$

At this point, the action $PickUp(Package, B)$ is not applicable because its preconditions are

$$At(Robot, B) \quad \text{and} \quad At(Package, B),$$

but the package is still at A .

Therefore, the example sequence in the laboratory sheet is not consistent with the stated initial state and action preconditions. A valid plan cannot be constructed from the given initial state using the listed actions, because there is no action that moves the package from A to B before it is picked up.

This was identified by explicitly checking the preconditions after each action rather than assuming that the example sequence was valid.

3 Task 2: Ask an LLM to Implement the Planner

A Large Language Model was given a precise specification of the planning problem rather than being asked to directly solve the problem. The specification required states to be represented as sets of logical propositions and actions to contain positive and negative preconditions and effects.

The planner was required to use breadth-first search (BFS) to find a sequence of applicable actions that achieves the specified goal. It was also required to detect when no plan exists, print the resulting action sequence, and print the states reached after each action.

Prompt Used

The following prompt was given to the LLM:

I want to implement a simple planning agent in Python for the warehouse planning problem.

Represent each state as a set of logical propositions.

Each action should contain:

- a name;
- positive preconditions;
- negative preconditions;

- positive effects;
- negative effects.

An action is applicable if all positive preconditions are satisfied by the current state and none of its negative preconditions are present.

When an action is applied:

1. remove its negative effects from the state;
2. add its positive effects to the state.

Use breadth-first search to find a sequence of actions that achieves a specified goal.

The warehouse has three locations: A, B, and C.

The robot and package initially start at A:

$$I = \{At(Robot, A), At(Package, A)\}$$

The goal is:

$$G = \{At(Package, C)\}$$

The robot can move between connected locations:

- Move(A,B)
- Move(B,A)
- Move(B,C)
- Move(C,B)

The robot can pick up the package when both the robot and package are at the same location.

The robot can drop the package at its current location.

The program should:

- represent states as sets of logical propositions;
- represent actions with positive and negative preconditions and effects;
- check action applicability before applying an action;
- use BFS to search for a plan;
- avoid repeatedly exploring the same state;
- detect when no plan exists;
- print the resulting sequence of actions;
- print the states reached after each action;

- explain the implementation and any assumptions.

Run the generated program on the warehouse problem.

Do not simply assume the program is correct. Verify that every action in the returned plan satisfies its preconditions and that the final state satisfies the goal.

Generated Planner

The LLM generated a Python planner based on logical states and breadth-first search. Each state is represented as a set of propositions, while each action stores its name, preconditions, and effects.

The function `is_applicable()` checks whether all the preconditions of an action are satisfied in the current state. The function `apply_action()` produces a successor state by removing the negative effects and adding the positive effects.

The planner uses a FIFO queue implemented using `deque` for BFS. A `visited` set is used to prevent the same state from being explored repeatedly. The search terminates when the goal propositions are contained in the current state.

When run on the warehouse problem, the generated planner produces the plan

PickUp(Package, A) → Move(A, B) → Move(B, C) → Drop(Package, C).

The final state contains

At(Package, C),

and therefore satisfies the specified goal.

Think About It

The preconditions are represented in the action objects and are checked by the `is_applicable()` function before an action can be executed. The effects are represented by the positive and negative effect sets and are applied by `apply_action()` to generate a successor state. The goal is checked during BFS, and planning terminates when the current state satisfies the goal. BFS is implemented using a FIFO queue, which causes alternative sequences of applicable actions to be explored level by level. The `visited` set prevents the planner from repeatedly exploring the same state.

4 Task 3: Test the Generated Planner

The generated planning program was tested using the three cases specified in the laboratory exercise. The purpose of the tests was to verify that the planner checks action preconditions correctly, detects impossible planning problems, and does not confuse irrelevant actions with actions that achieve the goal.

Test A: Solvable Problem

The original warehouse problem was used as the first test.
The initial state was

$$I = \{At(Robot, A), At(Package, A)\}$$

and the goal was

$$G = \{At(Package, C)\}.$$

The planner found the following plan:

PickUp(Package, A),
Move(A, B),
Move(B, C),
Drop(Package, C).

The states reached after each action were:

$$\begin{aligned} S_0 &= \{At(Robot, A), At(Package, A)\}, \\ S_1 &= \{At(Robot, A), Holding(Package)\}, \\ S_2 &= \{At(Robot, B), Holding(Package)\}, \\ S_3 &= \{At(Robot, C), Holding(Package)\}, \\ S_4 &= \{At(Robot, C), At(Package, C)\}. \end{aligned}$$

The final state satisfies the goal because it contains $At(Package, C)$. Each action in the returned plan also satisfies its preconditions when it is executed. Therefore, the returned plan is valid.

Test B: Impossible Problem

For the second test, the `PickUp` action was removed from the set of available actions. The initial state and goal remained

$$I = \{At(Robot, A), At(Package, A)\}$$

and

$$G = \{At(Package, C)\}.$$

Without a `PickUp` action, the robot cannot make the package enter the `Holding(Package)` state. Therefore, the package cannot be transported to C .

The planner reported:

No plan found.

This is the expected behaviour. The planner correctly reported that no plan exists rather than inventing an unavailable action.

Test C: Irrelevant Actions

For the third test, additional actions were introduced that could move the robot without moving the package. For example, the robot could move from A to B and from B to C while the package remained at A .

The initial state and goal were again

$$I = \{At(Robot, A), At(Package, A)\}$$

and

$$G = \{At(Package, C)\}.$$

The purpose of this test was to verify that reaching

$$At(Robot, C)$$

is not incorrectly treated as satisfying the goal

$$At(Package, C).$$

The planner must continue searching until the package itself reaches C . The resulting plan was checked by examining every state transition and verifying that the final state satisfies the package-location goal.

Summary of Testing

Test	Plan Found	Result	Plan Valid
Solvable problem	Yes	Package delivered to C	Yes
Impossible problem	No	No plan found	Yes
Irrelevant actions	Yes	Package delivered to C	Yes

Table 2: Results of testing the generated planning agent.

Testing Conclusion

The three tests were used to check different aspects of the generated planner. The solvable problem verified that the planner could construct a valid sequence of actions. The impossible problem verified that the planner could correctly report failure when an essential action was unavailable. The irrelevant-action test verified that the planner distinguished the robot's location from the package's location and did not incorrectly treat an irrelevant action as achieving the goal.

These tests demonstrate that a generated plan must be checked by examining the preconditions, effects, and resulting states rather than being accepted simply because the sequence of actions appears reasonable.

Think About It

The **preconditions** of an action are represented by its **positive_preconditions** and **negative_preconditions**. The function **is_applicable()** checks whether these preconditions are satisfied before an action is applied.

The **effects** are represented by the **positive_effects** and **negative_effects** of an action. The function **apply_action()** removes the negative effects from the current state and adds the positive effects to generate the successor state.

The **goal** is checked during BFS using the condition **goal <= current_state**. Planning terminates when all the goal propositions are contained in the current state.

BFS is implemented using a FIFO queue provided by **deque**. Applicable successor states are added to the queue and are explored level by level. A **visited** set prevents the same state from being explored repeatedly, allowing alternative plans to be searched systematically.

5 Task 4: Logic and Search

The planner combines two ideas: logical reasoning and search. Logical reasoning determines whether an action can be applied in the current state, while search determines which sequence of applicable actions should be explored.

For a state S and an action a , the logical component checks whether

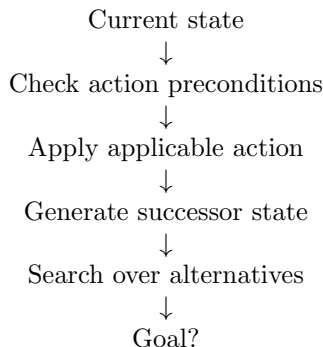
$$S \models \text{Preconditions}(a).$$

If all the required preconditions are satisfied, the action can be applied:

$$S' = \text{Apply}(S, a).$$

The resulting state S' is then used by the search component to explore alternative sequences of actions until a state satisfying the goal is found.

The planning process can therefore be described as:



In the Python planner, logical reasoning is implemented by **is_applicable()**, which checks whether the action's preconditions are satisfied. If an action is applicable, **apply_action()** updates the state according to its effects. The BFS

search then determines which applicable successor states should be explored and in what order.

Think About It

The planner can be understood as combining two complementary roles:

Logic determines what is possible; search determines what to try.

Logical reasoning restricts the search to actions whose preconditions are satisfied in the current state. Search then explores different sequences of these applicable actions until a sequence reaches the desired goal. Thus, logical reasoning determines the valid transitions between states, while search determines which transitions and sequences to explore.

6 Task 5: Can the LLM Verify Its Own Plan?

The generated plan was examined by asking the LLM to explain why each action in the plan is valid. The plan was

$$PickUp(Package, A) \rightarrow Move(A, B) \rightarrow Move(B, C) \rightarrow Drop(Package, C).$$

The preconditions and resulting states for each action are:

Action	Preconditions	Resulting State
<i>PickUp(Package, A)</i>	<i>At(Robot, A), At(Package, A)</i>	<i>At(Robot, A), Holding(Package)</i>
<i>Move(A, B)</i>	<i>At(Robot, A)</i>	<i>At(Robot, B), Holding(Package)</i>
<i>Move(B, C)</i>	<i>At(Robot, B)</i>	<i>At(Robot, C), Holding(Package)</i>
<i>Drop(Package, C)</i>	<i>At(Robot, C), Holding(Package)</i>	<i>At(Robot, C), At(Package, C)</i>

Table 3: Verification of the actions in the generated plan.

The independently executed Python planner produced the same state transitions. In particular, after the final action the state contained *At(Package, C)*, which satisfies the goal.

Which Should Be Trusted More?

The independently executed state transitions should be trusted more than the LLM’s explanation. The LLM explanation is a generated description of why the plan appears to be valid, whereas the Python program actually executes the precondition checks and state transitions.

An explanation generated by an LLM could contain a reasoning or description error even if the underlying program behaves correctly. Independent execution provides evidence based on the actual state transitions produced by the

planner. Therefore, the LLM explanation can be useful for understanding the plan, but it should not replace independent verification.

This illustrates the engineering principle:

A generated explanation is not the same as an independent verification.

7 Reflection Questions

1. Why is it useful to specify action preconditions and effects before asking an LLM to write the planner?

Specifying preconditions and effects gives the LLM a precise description of how actions can be applied and how they change the state. This reduces ambiguity in the implementation and provides a clear specification against which the generated program can be checked. In this laboratory, explicitly defining the preconditions and effects made it possible to verify whether each action in the generated plan was actually applicable.

2. Give an example of an error that could occur if the planner failed to check an action’s preconditions.

If the planner failed to check preconditions, it could apply an action when the required facts were not true. For example, it could execute `PickUp(Package,C)` even when the robot was not at *C* or the package was not at *C*. This would produce a state transition that is not valid according to the planning problem.

3. Why is a plan that “looks reasonable” not necessarily a valid plan?

A plan may appear reasonable as a sequence of actions while still violating the preconditions of one or more actions. A valid plan must ensure that every action is applicable in the state in which it is executed and that its effects are correctly applied. Therefore, the sequence must be checked through the actual state transitions rather than judged only by how plausible it appears.

4. What did the LLM contribute to the implementation?

The LLM contributed the concrete Python implementation of the planning agent based on the specified representation and requirements. It provided the action representation, precondition checking, state-transition logic, BFS search, and plan reconstruction. The generated implementation then had to be inspected and tested rather than being accepted without verification.

5. What did you have to verify independently?

The validity of the generated plan and the state transitions had to be verified independently. In particular, each action's preconditions had to be checked against the current state, the effects had to be applied correctly, and the final state had to satisfy the goal. The planner was also tested on solvable, impossible, and irrelevant-action cases.

6. In this laboratory, where is logical reasoning being used?

Logical reasoning is used when the planner determines whether an action is applicable by checking whether its preconditions are satisfied in the current state. It is also used when the effects of an applicable action determine the propositions that are removed from or added to the state. Thus, logic determines which state transitions are possible.

7. How is planning related to the search algorithms studied in the previous module?

Planning can be viewed as a search problem in which states are sets of logical propositions and actions generate successor states. The planner uses BFS to explore sequences of applicable actions until it reaches a state satisfying the goal. Thus, logical reasoning determines which actions are possible from a state, while the search algorithm determines which possible sequences should be explored.